

Roque Backend Stage Guide

How a plain-English instruction becomes a bounded on-chain action, how the two autonomous execution choices differ, and where Latch fits.

Prepared from the repository implementation and test suite
Revision: August 23, 2026 | Focus: backend, contracts, transaction flow



How to use this guide

This is a learning document for explaining Roque live. Read the first sentence of each section until the mental model is clear; use the implementation details when someone asks, “but how exactly?” The central idea is simple:

The agent can suggest and coordinate. The blockchain decides and moves value.

One-sentence answer: Roque uses GenLayer to turn natural language into a validated structured intent, then uses deterministic Solidity contracts on Sepolia to enforce permissions, limits, price conditions, and token movement.

The three distinctions you must keep separate

Choice	What it controls	Simple answer
Copilot / Autonomous	Who authorizes the action	You sign each action / the agent signs inside a capability.
Confirm / Direct	Whether autonomous waits for a review click	Show me the intent first / act as soon as it is interpreted.
Market swap / Limit order	When the swap settles	Now / later, after an on-chain trigger is true.

A useful custody sentence

The normal wallet is the user’s immediate account. The agent vault is a separate balance held by AgentExecutor. Autonomous trades can touch the vault, not the user’s main wallet. The user moves tokens into or out of the vault with their own signature.

What is actually verified in this checkout

- Solidity: 60 tests passed, including vault accounting, caps, slippage, expiry, revocation, nonce replay protection, and keeper-trigger checks.
- TypeScript core: 15 tests passed, including authenticated autonomous ownership, one-shot intent claiming, exact fixed-point cap boundaries, and EIP-712 recovery.
- GenLayer interpreter: 28 tests passed, including token normalization, malformed model output, exact-input enforcement, limit constraints, and adjudication defaults.

Stage discipline: describe what the code does today, then label future or architectural intent separately. In particular, the current Latch hook is an RPC egress proxy for relayer writes; it is not the on-chain capability registry.

1. The mental model

Roque is not “an AI wallet.” It is a normal exchange with a language interface and a bounded delegation layer. The language model is deliberately placed before the money-moving boundary.

Layer	Job	What it is not allowed to do
GenLayer intelligent contract	Read a sentence and produce a normalized intent; adjudicate fuzzy conditions.	Hold funds, sign Sepolia transactions, or bypass limits.
TypeScript core / API	Authenticate requests, call GenLayer, quote, preflight, sign autonomous intents, submit transactions, and record history.	Override the Solidity checks.
AgentExecutor	Hold the autonomous vault and enforce capabilities before agent actions.	Let an unapproved signer or over-cap action through.
DEXRouter + LiquidityPool	Quote and settle exact-input swaps against reserves.	Understand language or permissions.
OrderBook + keeper	Hold limit-order escrow and fill only when Chainlink says the trigger is met.	Force an untriggered fill.
Latch	Govern the off-chain path used to reach the RPC: credential boundary, rate limits, audit/revocation controls.	Replace on-chain authorization.
Neon Postgres	Store intent history, auth sessions, trade index, and indexer bookmark.	Be the source of truth for balances or caps.

The two trust boundaries

The first boundary is the GenLayer-to-Sepolia boundary. GenLayer may say “this reads like a swap,” but that statement is not a transaction. The relayer must turn it into a typed action, and the Solidity contract must independently accept it.

The second boundary is the off-chain-to-RPC boundary. The relayer and keeper are server processes with credentials and a gas-paying wallet. Latch can govern that route, but even a compromised relayer, keeper, or Latch path remains limited by the on-chain rules.

Why this matters: a model can be wrong without becoming a private-key theft incident. Its output still has to fit a structured message, recover to the authorized agent signer, pass the capability checks, and satisfy the market/order rules.

Repository map for speaking precisely

Path	What to point at
packages/genlayer/contracts/roque_interpreter.py	Natural language interpretation, deterministic validation, consensus comparison, subjective adjudication.
packages/core/src/services.ts	Prompt-to-intent orchestration, quotes, autonomous preflight, signing/submission.
packages/core/src/intents.ts	EIP-712 intent shapes, agent signing, executor calls, capability reads.
contracts/src/AgentExecutor.sol	Vault, capability, signature recovery, caps, slippage, nonces, autonomous settlement.
contracts/src/DEXRouter.sol + LiquidityPool.sol	Exact-input swap entry point and constant-product math.

Path	What to point at
contracts/src/OrderBook.sol + packages/core/src/keeper.ts	Escrowed limit orders, Chainlink trigger, public keeper execution.
packages/core/src/chain.ts + env.ts	Relayer RPC transport and optional Latch proxy routing.
packages/core/src/indexer.ts + db/index.ts	On-chain event mirror and off-chain audit/history state.

2. From prompt to structured intent

The first phase does not trade. It interprets. This is important because the user should be able to see or review what Roque thinks they asked for before a transaction is built.

Step 1: the request enters the shared backend

The browser sends the command to the API. Both the standalone Fastify service and the Next.js routes call the same functions in @roque/core, so there is one implementation of the business rules.

The service creates a UUID and inserts an intents row with the original text, wallet address if known, mode, and status interpreting. This database row is a notebook entry, not permission.

Step 2: the backend sends the sentence to GenLayer

The TypeScript client submits a write to the deployed RoqueInterpreter contract. The write is needed because the interpreter stores the result under the request ID. The client waits for the GenLayer receipt to finalize, then reads the stored result back.

On GenLayer, the contract builds a strict prompt containing the ten canonical tokens, the user's sentence, and optional context such as ETH/USD. A validator runs the non-deterministic prompt function. The equivalence principle compares normalized JSON answers rather than raw prose, so validators can disagree on wording but must agree on the actual trade fields.

Step 3: deterministic validation cleans up the model output

- Token aliases are normalized: ETH becomes rWETH, BTC becomes rWBTC, GOLD becomes rPAXG, and so on.
- Only the ten deployed tokens are accepted.
- Input and output must differ.
- Amounts must be plain positive decimal strings; percentages must be between 0 and 100.
- Roque is exact-input: "spend 5 rWBTC" is valid; "receive exactly 20 rUSDC" is rejected because the contracts do not solve an exact-output swap.
- A limit must include rWETH because the deployed OrderBook watches ETH/USD, not every token's price.
- Malformed JSON, prose, or validator disagreement becomes a refusal, not an executable trade.

Step 4: the backend stores and quotes the interpretation

A successful result is saved with kind, tokenIn, tokenOut, amount, trigger fields, confidence, reason, and the full JSON interpretation. For market swaps, the backend calls the same deployed router's

quoteSwap view. That view reads the matching LiquidityPool reserves and applies the pool fee.

For a limit order, the fill quote is not final yet. The output floor is constructed for the order and checked at fill time by the router/pool. The preview is a notional explanation, not a guarantee that the future pool state will be identical.

```
Prompt: swap 200 USDC for ETH Stored: kind=swap, tokenIn=rUSDC, tokenOut=rWETH, amount=200 Quote:
router.quoteSwap(rUSDC, rWETH, 200 units) Next: Copilot -> user wallet; Autonomous -> agent capability
path
```

What to say on stage

“GenLayer is the judgment layer, not the custody layer. It translates English into a structured object, and then Roque treats that object as untrusted input until the deterministic execution layer accepts it.”

3. Copilot mode: the user signs

Copilot is the familiar DEX path with a natural-language front door. The user stays in control of every action and no agent capability is needed.

Market swap: exact flow

Moment	What happens
1. Interpret	The prompt is stored and interpreted by GenLayer. The result says what token is being spent, what token is received, and how much input to spend.
2. Prepare	The backend calls router.quoteSwap and calculates minAmountOut from the selected slippage tolerance.
3. Approve	The user wallet gives the Router an allowance for exactly the input amount if the current allowance is insufficient. There is no unlimited approval.
4. Sign and send	The user wallet sends DEXRouter.swapExactTokensForTokens(tokenIn, tokenOut, amountIn, minAmountOut, user, deadline).
5. Router pulls input	The Router pulls tokenIn from the user, approves the selected pool, and calls LiquidityPool.swap.
6. Pool computes output	The pool applies constant-product pricing and a 0.30% fee, checks minAmountOut, transfers tokenOut to the user, and syncs reserves.
7. Land	The output token is already in the user’s normal wallet because the router’s recipient is the user address.
8. Record	The client attaches the tx hash to the intent. The indexer later records the RouterSwap event in Neon for history.

What the contract checks

- The deadline has not passed.
- The pair has a registered pool.
- The user approved enough tokenIn.

- The pool has enough liquidity.
- The calculated output is at least `minAmountOut`.

Limit order: exact flow

The user signs `OrderBook.createOrder`, not a swap. The input tokens are transferred into the `OrderBook` immediately and become escrow. The order stores owner, pair, amount, min output, trigger price, direction, expiry, and status `Open`.

The order is not self-executing. A keeper polls order IDs, asks the contract's `isTriggered` view whether the ETH/USD condition is true, and calls `executeOrder`. The contract reads Chainlink again inside the transaction. If the price is wrong, stale, or the order expired, it reverts.

When the trigger is true, `OrderBook` marks the order `Filled`, approves the Router, and the Router sends the swap output directly to the order owner's wallet. The keeper is only paying gas; it cannot choose an unearned fill.

Important distinction: a limit order is “escrow now, swap later.” The input leaves the wallet when the order is created; the output arrives when the trigger is verified and the order fills.

4. Autonomous mode: bounded delegation

Autonomous mode removes the wallet popup from each trade, but it does not give the agent the user's wallet key. The user signs a capability once. That capability says what the agent may do, how much it may spend, and when the permission expires.

First, the user creates the working balance

The user approves `AgentExecutor` for an exact amount and calls `deposit`. `AgentExecutor` transfers the tokens into its own contract address and increments `vaultBalance[user][token]`. The user's main wallet no longer holds those deposited units; the executor contract does.

The agent cannot withdraw from the vault. Only the user can call `withdraw`, which sends tokens back to the user's normal wallet. This is the core custody boundary.

Then, the user grants a capability

The grant is EIP-712 typed data. It names the user, the fixed `agentSigner` address, `maxPerTradeUsd`, `maxDailyUsd`, `maxSlippageBps`, `validUntil`, and a `grantNonce`. The user can submit it directly with `grantCapability`, or sign it off-chain and let the relayer submit `grantCapabilityWithSig` to pay gas.

The relayed grant is not trusted merely because the relayer sent it. `AgentExecutor` reconstructs the typed-data digest and recovers the signer. It must recover to the user address, and the grant nonce is incremented so the same grant signature cannot be replayed.

Capability field	Meaning
<code>agentSigner</code>	Only this Ethereum address may sign autonomous actions for this user.

Capability field	Meaning
maxPerTradeUsd	Dollar value ceiling for one action, using executor pricing.
maxDailyUsd	Total dollar value allowed for the current UTC-like on-chain day bucket.
maxSlippageBps	The worst output floor the agent is allowed to request relative to the current router quote.
validUntil	Hard expiry. After this timestamp, the capability is dead.
revoked	Immediate user-controlled kill switch.
grantNonce	Prevents replaying an old capability grant.

Say this: “The capability is not a private key. It is a contract-enforced allowance with a signer identity, numeric limits, and an expiry.”

Wallet authentication is a separate server concern

Before autonomous API requests, the user signs a one-time message saying this is for Roque autonomous access and does not authorize a trade. The server verifies the message, atomically consumes the challenge, and stores only a hash of a short-lived bearer token. That session proves which vault owner is making the request; it is not the on-chain grant.

5. The two autonomous execution choices

The implementation has two autonomous timing choices. They use the same backend and the same on-chain enforcement. The only difference is whether the frontend pauses after interpretation.

Confirm mode

The user writes a command. GenLayer interprets it. The UI displays the structured intent and quote. When the user clicks the action, the browser authenticates the owner session and calls the execute API. The server signs and submits the action.

Confirm mode is useful when the user wants autonomous custody boundaries but still wants to catch a misread sentence, wrong token, wrong amount, or wrong trigger before it is sent.

Direct mode

The user writes a command. Once GenLayer returns an executable interpretation, the frontend immediately calls the same execute API without waiting for a card tap. It still requires a live capability, still uses the authenticated owner session, and still passes through the exact same TypeScript preflight and Solidity checks.

Direct mode changes the human review point, not the authority model. It is faster and more dangerous because a mistaken interpretation becomes a real attempt immediately. The on-chain cap is still the last word, but it cannot tell whether the user intended rWETH or rWBTC if both are allowed and within the cap.

Question	Confirm	Direct
Does the UI show the interpreted intent first?	Yes	No pause after a valid interpretation
Does the user sign each autonomous trade?	No	No
Does the agent sign each action?	Yes	Yes
Do caps, expiry, vault, nonce, and slippage checks change?	No	No
What risk changes?	Human catches more interpretation errors	More speed; less time to catch a mistaken request

Autonomous market swap: prompt to vault

Step	Backend and contract behavior
1. Owner check	The server verifies the wallet session and confirms the stored intent belongs to that same vault owner and is autonomous.
2. Capability read	The server reads the current capability and refuses early if missing, revoked, expired, or naming a different agent signer.
3. Amount resolution	A percentage is calculated from the user's vault balance, not the main wallet. A concrete amount is parsed using token decimals.
4. Vault and cap preflight	The server reads vault balance and calls AgentExecutor.usdValue for exact fixed-point USD math. It rejects insufficient funds or over-cap trades before signing.
5. Quote and floor	The server reads the router quote and computes minAmountOut using the smaller of the user's requested slippage and capability maxSlippageBps.
6. One-shot claim	A conditional database update changes intent status from ready to signed. This prevents two concurrent API calls from signing the same stored intent.
7. Agent signature	The agent key signs the exact EIP-712 SwapIntent fields: user, token addresses, amountIn, minAmountOut, nonce, deadline.
8. Relayer transaction	The gas-paying relayer sends AgentExecutor.executeSwap(intent, signature).
9. On-chain authorization	AgentExecutor checks deadline, token registration, capability, expiry, recovered signer, unused nonce, per-trade cap, and daily cap.
10. On-chain settlement	It checks slippage, debits the input from the vault, routes the swap, and credits output to the same user's vault balance.

Where the swapped token lands: for an autonomous market swap, output is credited to `vaultBalance[user][tokenOut]`. It does not automatically appear in the normal wallet. The user must call `withdraw`, with their own wallet signature, to move it out of the vault.

6. Autonomous limit order: prompt to eventual wallet payout

An autonomous limit order has two transactions separated in time. The first transaction opens and escrows the order. The second transaction fills it later when the chain verifies the trigger.

Phase A: opening the order

- The prompt is interpreted as a limit only if rWETH is one side and a positive ETH/USD trigger is present.
- The server resolves the amount, checks vault balance, values the input for the per-trade cap, and chooses a fresh nonce.
- The agent signs a LimitIntent containing token pair, amount, min output, trigger price, direction, order expiry, nonce, and short submission deadline.
- The relayer submits AgentExecutor.createLimitOrder.
- AgentExecutor runs the same capability checks as a swap, then debits the input from the vault.
- AgentExecutor approves OrderBook and calls createOrderFor. OrderBook pulls the escrowed input from AgentExecutor and records the order owner as the user.

Phase B: waiting and filling

- The OrderBook holds the input tokens in escrow. They are no longer in the user wallet or the agent vault.
- The keeper scans order IDs every 15 seconds in the long-lived worker, or the deployment can nudge the cron route.
- The keeper calls isTriggered, but this is only an optimization. executeOrder checks again inside the transaction.
- OrderBook reads Chainlink ETH/USD and rejects bad, stale, expired, or untriggered orders.
- If true, the order becomes Filled before the router call. The Router swaps through the matching pool and sends tokenOut to the order owner.
- The output therefore lands in the user's normal wallet for the current autonomous limit-fill implementation.

Cancel path

The user can cancel an open order with their wallet. OrderBook marks it Cancelled and returns the escrowed input to the owner. AgentExecutor is also allowed to cancel an order it opened for that user. A random keeper cannot cancel someone else's order.

Why a malicious keeper cannot steal a fill

The keeper chooses when to spend gas, but it does not choose whether the order qualifies. The contract reads the price feed and compares it to the stored trigger. A dishonest keeper can submit a doomed transaction or race another keeper, but cannot make the contract fill an untriggered order.

One subtle answer: the limit order's trigger is ETH/USD even when the other leg is another supported token. This is why the interpreter rejects a limit request with no rWETH. Market swaps work across the deployed token mesh; limits are intentionally narrower.

Keeper timing note

The worker implementation runs every 15 seconds. The repository also has a GitHub Actions keeper workflow every five minutes and a Vercel cron configuration. When presenting latency, say “the keeper is scheduled/pollled; it is not block-native or self-executing,” and distinguish the worker interval from the deployment scheduler.

7. What happens inside a market swap

The Router and Pool are deliberately boring. That is a feature. They do not know about GenLayer or Latch; they only implement deterministic token movement and pricing.

The quote

Each pair has one LiquidityPool. The deployed mesh contains 45 unordered pairs for the ten tokens, so a supported pair swaps in one hop. The Router finds the pool using an order-independent pair key.

For input amount x , input reserve X , output reserve Y , and fee-adjusted input $x' = x \times (10,000 - \text{feeBps})$, the pool computes:

$$\text{amountOut} = (x' \times Y) / (X \times 10,000 + x') \text{ where } \text{feeBps} = 30$$

This is constant-product AMM behavior. Larger trades move the reserves more and receive a worse marginal rate. The quote is a snapshot; another transaction can change reserves before the user's transaction lands, which is why `minAmountOut` exists.

The slippage floor

The user chooses a tolerance such as 1% = 100 basis points. The backend takes the quote and computes $\text{minAmountOut} = \text{quote} \times (10,000 - \text{slippageBps}) / 10,000$. If the pool would return less than that, the transaction reverts instead of delivering an unexpectedly bad price.

The autonomous slippage rule

The autonomous path uses the tighter limit: the user's requested slippage or the capability's `maxSlippageBps`. In plain terms, the agent cannot silently choose a looser floor than the permission allows.

The dollar caps

`AgentExecutor` values the input token. Stablecoins are treated as one dollar. Non-stables use their registered Chainlink feed and normalize token decimals and feed decimals into an 18-decimal fixed-point dollar value. That same `usdValue` view is used by the TypeScript preflight, avoiding a JavaScript rounding mismatch.

The daily cap is stored by day bucket: `block.timestamp / 1 days`. Each successful authorization adds to that day's spent total. It is a per-user, per-day accounting bucket, not a rolling 24-hour window.

Answer to “what if the agent key is stolen?” The thief can sign messages as the agent, but cannot change the user field, exceed the vault balance, exceed the per-trade or daily cap, use a revoked/expired capability, reuse a nonce, or ask for a worse-than-allowed slippage floor. The remaining risk is whatever the user allowed inside that box.

8. Latch: what it does and what it does not do

Latch is an off-chain control point around the machinery that submits Sepolia writes. It is not the contract that authorizes a user's trading capability, and it does not hold the user's vault funds.

Current repository behavior

The relayer's `WalletClient` normally sends JSON-RPC requests directly to `SEPOLIA_RPC_URL`. When both `LATCH_RPC_URL` and `LATCH_TOKEN` are configured, the relayer's write transport points to the Latch URL and sends `Authorization: Bearer lat_...`

The private relayer signing key is still loaded by the application and used to sign the Ethereum transaction locally. Latch sits on the network egress path for that wallet client. The read client remains direct to the Sepolia RPC. In the current code, relayed capability grants, autonomous agent-intent submissions, and keeper order fills all use this relayer wallet path.

Layer	Question	Answer
User wallet	Who sets the capability?	The user, by direct transaction or EIP-712 signature.
AgentExecutor	What can the agent do?	Only the typed actions accepted by the contract, within on-chain capability and vault limits.
Latch	Can the relayer/keeper reach the RPC?	It can govern that off-chain route when configured: credential indirection, scoped token, rate/routing policy, audit/revocation boundary.
Sepolia	What finally decides?	The deployed contracts. If Latch or the relayer submits an invalid action, the transaction reverts.

How to explain the value of Latch

- It gives the off-chain signer/RPC channel a separate policy boundary from the application process.
- The raw provider credential can be kept behind the proxy rather than placed directly in the relayer environment.
- The channel can be scoped, rate limited, disabled, and audited independently of the app's business logic.
- A compromised Latch path still cannot grant itself a larger on-chain capability or bypass AgentExecutor.

Do not overclaim

- Do not say Latch is the source of truth for funds. AgentExecutor and OrderBook are the source of truth for autonomous balances and orders.
- Do not say Latch makes the relayer trustless. The relayer remains an off-chain service.
- Do not say Latch replaces EIP-712. EIP-712 is the cryptographic authorization format used by the user grant and agent intents.
- Do not imply the current checked-in code proves TEE custody or attestation. The repository architecture describes those as a Latch phase/goal; the concrete hook here is proxy routing via environment variables.

Stage answer: “Latch is the second fence around the off-chain machinery. The first fence is the on-chain capability and vault. Latch can stop or constrain the signer/RPC channel, while Sepolia remains the final authority over token movement.”

9. Authentication, replay protection, and history

Wallet-bound autonomous access

The server does not trust a caller just because the caller supplies a wallet address. It issues a challenge containing the address, chain ID, random nonce, issue time, and expiry. The wallet signs it. The server verifies the signature, consumes the challenge with a conditional update, and creates a one-hour session.

Only a SHA-256 hash of the bearer token is stored in Postgres. A database reader cannot simply copy a live token because the raw token is not stored. The session is used to prove that the API caller owns the vault referenced by an autonomous intent.

Replay protection exists at more than one layer

- Wallet auth challenge: one-time consumption and five-minute expiry.
- Grant signature: grantNonce increments on-chain.
- Agent action: user-scoped intent nonce is checked in AgentExecutor.usedNonce.
- Backend intent: the database changes status from ready to signed only once.
- Frontend action card: once a transaction is submitted or fails after submission, it does not offer the same action again.
- Order status: Open can transition to Filled or Cancelled, and a second fill reverts.

What Neon is for

Neon stores the original prompt, interpretation, status, transaction hash, auth sessions, trade rows, and the indexer bookmark. It makes the activity feed searchable and lets the backend coordinate a user request. It does not hold the vault, calculate authoritative caps, or determine whether an order is triggered.

How the indexer avoids lying or double-counting

The indexer starts from a deployment block, scans bounded block windows, writes rows keyed by transaction hash and log index, and advances a bookmark only after a window is processed. It reads AgentSwap for autonomous market swaps, RouterSwap for direct wallet swaps, and OrderCreated/OrderFilled/OrderCancelled for limit lifecycle events.

The database is therefore a projection of chain events. If the UI and database disagree with the chain, Etherscan and the deployed contract state win.

Failure and recovery

Failure	What happens
GenLayer unavailable	Interpretation fails or retries; normal DEX functions can still work because contracts and copilot are independent.
RPC hiccup	GenLayer and Neon calls have bounded retries; worker loops log and retry on the next tick.
Relayer offline	Autonomous submissions and keeper fills wait. Existing on-chain orders and caps are not erased.
Keeper races another filler	One transaction wins; the other reverts harmlessly because status is no longer Open.
User revokes capability	Future agent actions fail on-chain immediately after the revoke transaction mines.
User cancels limit order	Escrowed input returns to the user; the order becomes Cancelled.

10. Questions you may be asked

“Why not just call an LLM from the backend?”

Because the product wants a judgment step whose result is consensus-checked and auditable. GenLayer is used for language and subjective interpretation. It still cannot move money; the deterministic Sepolia contracts are the financial source of truth.

“Can GenLayer directly send the Ethereum transaction?”

No. In this design, GenLayer reads/stores its own decision. It does not possess the Ethereum private key or a native way to produce a Sepolia signature accepted by AgentExecutor. The off-chain relayer bridges the finalized decision into a typed, signed Sepolia transaction.

“Does the model choose the price?”

For a market swap, the Router/Pool determines output from reserves. For autonomous caps, AgentExecutor values input using stablecoin rules or Chainlink feeds. For a limit trigger, OrderBook reads Chainlink ETH/USD itself. The model may parse a user-specified trigger, but it cannot make the contract accept an untrue condition.

“What exactly is signed?”

In copilot, the user wallet signs the actual Router or OrderBook transaction. In autonomous, the user signs the capability grant once, then the agent signer signs each typed SwapIntent or LimitIntent. The relayer’s own wallet pays gas and submits the signed call to AgentExecutor.

“Can the relayer spend from the user’s main wallet?”

Not through autonomous execution. The executor only debits the user’s vault balance. The relayer’s Ethereum account is a gas payer and transaction submitter, not the owner of the user’s wallet. Copilot is sent by the user’s own wallet.

“Where do autonomous swap proceeds go?”

Autonomous market swap proceeds are credited to the user’s vault. The user must withdraw them to the normal wallet. Autonomous limit-fill proceeds currently go to the order owner’s normal wallet because OrderBook passes the owner as the Router recipient.

“Can the user withdraw while the agent is trading?”

The user can call withdraw, but it can only withdraw the current vault balance. If a pending agent transaction later reaches the chain and the balance is insufficient, the transaction reverts. The vault and executor checks remain authoritative.

“What is the biggest limitation?”

This is a testnet demo with shallow pools and centralized off-chain services. The AMM spot price can be moved by trades, the keeper/relayer are not decentralized, GenLayer-to-Sepolia writes are off-chain-relayed, and the deployed OrderBook trigger is specifically ETH/USD. The design bounds those risks; it does not make them disappear.

“What happens if the model hallucinates?”

First, the GenLayer deterministic gate can reject unknown tokens, invalid amounts, exact-output requests, and incoherent limits. If a bad but structurally valid intent still gets through, the user review in Confirm mode can catch it. If Direct mode is enabled, the on-chain capability, vault, signature, cap, slippage, expiry, and nonce checks still bound the action. The system does not promise semantic perfection; it limits the financial consequences.

Short closing answer: “Roque separates interpretation from authorization and execution. GenLayer helps understand the user. The backend coordinates. Latch governs the off-chain route. But only deterministic Sepolia contracts can move the tokens.”

11. Honest implementation notes

These are the details that distinguish a strong technical presentation from a marketing summary. They are not all exploits; many are MVP tradeoffs or places where the architecture document describes a broader target than the current code implements.

Capability scope is numeric, not per-token

The architecture text talks about allowed tokens and actions. The current Capability struct contains agentSigner, per-trade cap, daily cap, slippage cap, expiry, revoked, and exists. It does not contain a user-specific token allowlist or action bitmask. Token scope is the global executor registry, and action scope is the fixed typed entry points: executeSwap and createLimitOrder.

Limit-order slippage is less strongly enforced than market-swap slippage

AgentExecutor calls its capability slippage check for executeSwap, but not for createLimitOrder. The normal backend computes a minAmountOut using the capability limit, and OrderBook enforces that stored floor at fill. However, a compromised agent-signing path that bypasses the normal backend could sign a limit intent with a very low floor. A production hardening would enforce the capability's limit-order floor on-chain as well.

Existing limit orders outlive capability changes

The capability is checked when the autonomous order is opened. Once input is escrowed in OrderBook, a later revoke or capability expiry does not automatically cancel the order and is not rechecked at fill. The user must cancel the open order if they want the escrow returned and the future fill stopped.

Daily accounting is a day bucket

The cap resets when the on-chain day index changes, not exactly 24 hours after each trade. Opening a limit order consumes daily headroom immediately. Cancelling it does not refund that day's accounting, because the daily total records authorized action volume rather than net settled exposure.

The two limit builders are not identical

Autonomous service code starts minAmountOut from the current AMM quote. Copilot browser code builds a limit floor from oracle values at the trigger price. Both floors are enforced by the Router at fill, but they model the future price differently. This is an implementation inconsistency worth knowing if a demo order refuses to fill.

Broadcast is not always confirmation

Browser-signed copilot writes wait for a Sepolia receipt before returning. The autonomous relayer returns the transaction hash after sendTransaction and does not wait for a receipt in that request. The indexer later supplies chain-confirmed history. So "submitted" and "mined" are distinct states even if the UI uses optimistic language.

Operational edge cases

- If autonomous signing or submission fails after the database claims the intent as signed, the same stored intent is not retried; the user creates a new request.
- Fresh action nonces start from millisecond time and are checked on-chain. Two different intents racing in the same millisecond could collide; one would revert rather than replay.
- Expired limit orders do not auto-refund. They can no longer fill, but the owner must cancel to reclaim escrow.
- OrderBook's owner can change the price feed, staleness threshold, and agentExecutor address; Router and AgentExecutor also have owner-managed registries. This is admin trust in the MVP.
- The test tokens are centrally mintable by their owner and exist for the Sepolia demo; they are not production assets.

- The default OrderBook permits a Chainlink answer up to 24 hours old, deliberately generous for a quiet testnet.

How to present these: “The security model is real, but this is still an MVP. The on-chain caps and custody separation are the strongest part. Production hardening would tighten limit-order capability enforcement, admin controls, nonce allocation, confirmation tracking, and Latch key-custody integration.”

12. The 90-second explanation

Roque is an agent-native exchange on Ethereum Sepolia. A user can type a command such as “swap 200 USDC for ETH.” The command is stored and sent to a GenLayer intelligent contract. GenLayer uses validator agreement to interpret the sentence into a structured intent, then deterministically rejects unknown tokens, invalid amounts, and unsupported order types.

In Copilot mode, the result is only a preview. The backend reads a live quote from the deployed router and computes a slippage floor. The user’s wallet approves the exact input amount if needed and signs the router transaction. The pool applies constant-product math and sends the output directly to the user’s wallet.

In Autonomous mode, the user first deposits funds into a separate AgentExecutor vault and signs an EIP-712 capability: agent address, per-trade cap, daily cap, slippage limit, and expiry. For each action, the agent signs a typed intent. The relayer submits it, but AgentExecutor recovers the signer and independently checks every bound before moving a token.

Autonomous Confirm mode shows the interpreted action before the execute call. Direct mode sends it immediately after interpretation. Both use the same enforcement. A market swap credits the output to the user’s vault; the user withdraws it with their own wallet. A limit order escrows input in OrderBook, waits for Chainlink ETH/USD to satisfy the trigger, and can be filled by any keeper. The contract rechecks the trigger, so the keeper cannot force a bad fill.

Latch is the off-chain second fence. When configured, the relayer’s RPC writes go through a scoped Latch proxy instead of directly to the provider. That gives the off-chain signing path a separate credential, policy, rate, and audit boundary. Latch does not replace the on-chain capability system. Sepolia remains the final authority.

Final memory hook

Interpretation is flexible. Authorization is typed. Execution is deterministic.

That is the sentence to return to whenever a question gets complicated.

13. Current deployment snapshot

These addresses come from the current checked-in deployment metadata (packages/shared/src/deployment.json and packages/genlayer/deployment.json). Use them when someone asks where the live demo is deployed. The repository README contains an older

address table, so the deployment JSON is the better source for this checkout.

Network	Component	Address
Ethereum Sepolia chainId 11155111	AgentExecutor	0xff3ACF2377C831886bF674543eb4bE38DF19a5cc
Ethereum Sepolia	DEXRouter	0x7965E72630cDBC3d0cc0D6DdC75497d674d49799
Ethereum Sepolia	OrderBook	0x04d38a17587B4F7Ba9c477857735475aD39c61B4
Ethereum Sepolia	ETH/USD Chainlink feed	0x694AA1769357215DE4FAC081bf1f309aDC325306
Ethereum Sepolia	Agent signer address	0x1254F7FBc7Cb0289c06a649A032783e7d0C68934
GenLayer Studio Network	RoqueInterpreter	0x0A30a949542A7EDE8a546D6Ad2C55738e8146573

Token roster

The current deployment contains ten Roque test tokens: rUSDC, rUSDT, rDAI, rWETH, rWBTC, rLINK, rSNX, rFORTH, rEURC, and rPAXG. There are 45 pair pools, one for each unordered pair. rUSDC, rUSDT, and rDAI are treated as one-dollar stables for executor cap valuation; the other tokens use their registered Chainlink feeds.

A final caveat on addresses

Addresses identify the current deployment, not a universal protocol identity. A redeploy changes them, and the frontend/backend consume the generated deployment metadata so the contracts, ABIs, and EIP-712 verifying contract stay aligned.